DisputeDesk — Phase 3 Report

Database persistence: accounts, demo disputes, evidence, sessions and an audit log in SQLite

Date **3 November 2026** Commit **5145cbe** (not pushed) Builds on **bd1f620** (Phase 2.1/2.2) Branch **master**

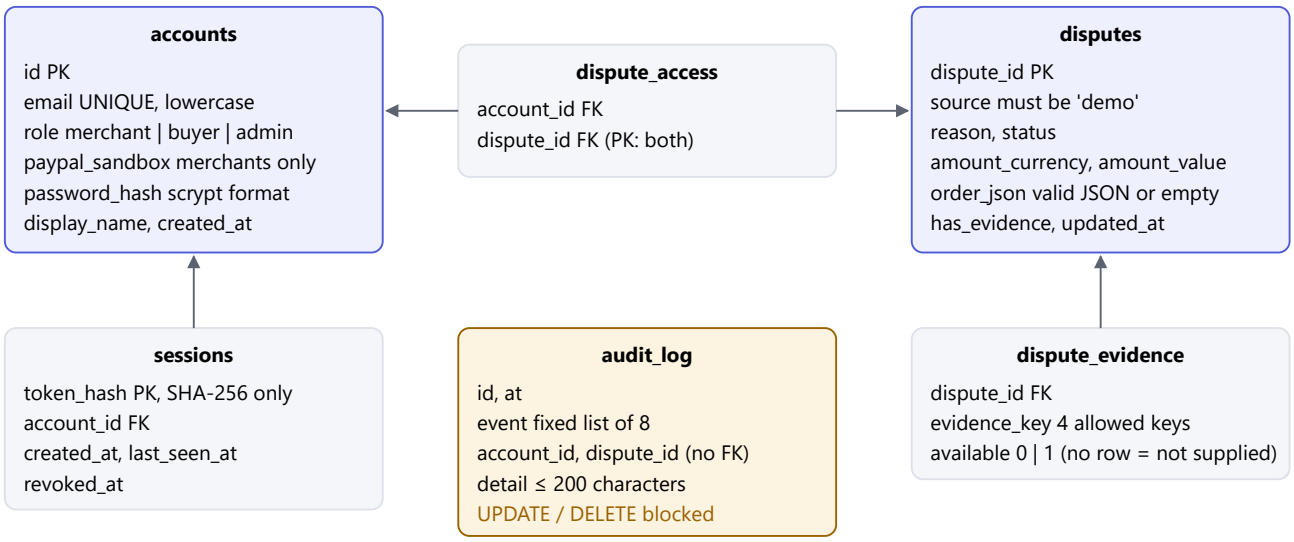
Summary. DisputeDesk now keeps its data in a SQLite database through Node's built-in `node:sqlite`, so no dependency was added. Sessions survive a server restart while keeping the 30-minute idle and 8-hour limits. Important security and dispute actions go to an append-only audit log. Every API response, role rule, cookie, cross-site check and security header is unchanged. All 113 earlier server tests still pass, alongside 26 new ones. No PayPal or Gemini calls were made.

139/139	53/53	22/22	22/22	0
Server tests (26 new)	Client tests	Browser checks, production build	Browser checks, dev server	New dependencies or secrets

1. Before and after

Area	Before (Phase 2)	After (Phase 3)
Sessions	In memory; a restart signed everyone out	In the database; survive restarts; expiry and sign-out unchanged
Accounts	Read from `demo-accounts.json`	Stored with constraints; mirrored from the file on start
Demo disputes and evidence	Read from the fixture files	Stored in tables; read back exactly as the files
Dispute access	`dispute_ids` list in the JSON	`dispute_access` table with foreign keys
Audit trail	None	Append-only `audit_log`
PayPal sandbox	Read live, read-only	Unchanged: read live, never stored

2. Database schema (migration v1)



Schema v1. Arrows are foreign keys; deleting an account or dispute removes its access, evidence and sessions. The audit log has no foreign keys on purpose, so entries outlive the records they mention.

Table	Constraints	Indexes
`schema_migrations`	One row per applied migration (version, name, time)	Primary key
`accounts`	id: lowercase letters, digits, hyphens, at most 40; unique lowercase email containing @; role from 3 values; sandbox only for merchants; scrypt hash; display name not blank	Primary key, unique email
`disputes`	`source = 'demo'` only; amount currency and value both present or both empty; order stored as valid JSON	Primary key
`dispute_evidence`	Key from the 4 evidence types; value 0 or 1; linked to its dispute	Primary key (dispute, key)
`dispute_access`	Both links checked; removed with the account or dispute	Primary key, `idx_dispute_access_dispute`
`sessions`	Key is exactly 43 characters (a SHA-256 hash); linked to an account; last use not before creation	`idx_sessions_account`, `idx_sessions_created`
`audit_log`	Event from a fixed list; detail at most 200 characters; triggers refuse UPDATE and DELETE	`idx_audit_account_at`, `idx_audit_event_at`

All tables are `STRICT`, so SQLite rejects values of the wrong type. Foreign keys are switched on for every connection, the file database uses WAL mode, and a 5-second busy timeout applies.

3. Migrations and transactions

- **Numbered and forward-only.** Migrations live in `server/src/db.ts`. A released migration is never edited; a change is a new migration at the end. A test checks the versions run 1, 2, 3… with no gaps.

- **One transaction each.** A failing migration leaves nothing half-applied, and the server does not start.
- **Newer database refused.** A database written by a newer server stops startup instead of being misread.
- **Atomic writes.** `transaction()` commits every write or none. It covers sign-in (new session plus audit entry), sign-out, account creation (account plus its access) and the demo mirror on start.

4. Sessions

Rule	How
Token never stored	The row key is the token's SHA-256 hash; a test checks the raw token is nowhere in the table
30 minutes idle, 8 hours absolute	Checked against `last_seen_at` and `created_at` on every request
Sign-out	Sets `revoked_at`; a revoked session is refused, also after a restart
New sign-in	Revokes the browser's previous session and issues a new token
Clean-up	Expired and revoked rows are deleted at the next sign-in
Password change	A changed password hash in the demo file ends that account's sessions
Cookie	Unchanged: `__Host-dd_session; Path=/; Max-Age=28800; HttpOnly; Secure; SameSite=Strict`

5. Audit log

Event	Written when	Records
`login_succeeded`	Sign-in works	Account
`login_failed`	Wrong password or unknown email	Account if known; nothing for an unknown email
`login_locked`	Sign-in refused by the attempt limit	Account if known
`logout`	A valid session signs out	Account
`dispute_access_denied`	An account asks for another account's dispute	Account, dispute, source
`analysis_viewed`	Evidence analysis returned	Account, dispute, source
`ai_draft_generated`	Checked AI draft returned	Account, dispute, source and model
`ai_example_viewed`	Validation example returned	Account, dispute, source

Never recorded: passwords, session tokens, cookies, password hashes or unknown emails. A test checks this, and a live check of the real database found no demo password in the audit or session tables.

6. Security decisions

- **Parameterized SQL only.** Every query uses `?` placeholders; no value is ever put into SQL text.
- **Server-side identity.** The account and role come from the session row, never from the request.
- **Same isolation.** Merchants and buyers reach only disputes in `dispute_access`; another account's dispute answers 404, identical to a missing one, and the attempt is audited. Buyers still get status and amount only.

Admin is read-only and has no write route.

- **Demo data stays labelled and separate.** The table only accepts `source = 'demo'`. The demo files remain the source of truth. PayPal sandbox disputes are never stored.
- **No real customer data.** All accounts are the five synthetic demo accounts; there is still no sign-up route.
- **Safe failures.** A database error returns a generic 500 with no database details, and no cookie is issued. The cause goes to the server log only.
- **File protection.** `server/data/` is gitignored; it holds password and session hashes only.

7. Tests and verification

Area	New tests	Examples
Database and migrations (`db.test.ts`)	9	Fresh migration, re-run is a no-op, data kept after reopening a file, newer database refused, failed migration rolled back, transactions all-or-nothing, constraints, foreign keys, append-only audit log, indexes present
Store (`store.test.ts`)	8	Disputes and evidence read back exactly as the files, missing evidence stays missing, accounts by id and email, created account survives reopening, failed account creation rolled back, demo mirror, failed mirror changes nothing, audit rows
Sessions (`auth.test.ts`)	4	Survive restart, only the hash stored, revoked stays revoked, clean-up, no session for an unknown account
App (`app.test.ts`)	5	Restart keeps sign-in and sign-out ends it everywhere, audit events and no secrets, AI draft and lockout audited, database failure gives a generic 500, failed sign-in leaves no session or audit row
Existing tests	113 kept	Role matrix, cross-account 404s, buyer view, cookie, CSRF, headers. Only their session setup changed.

Check	Result
Server tests	139/139
Client tests	53/53
Server and client type checks	Pass
Production build	Pass
Browser checks, production build (`localhost:4173`)	22/22
Browser checks, dev server (`localhost:5173`)	22/22
Live restart: signed in, API restarted, `/api/auth/me`	200, then 401 after sign-out
Mutation checks: transaction removed; a fresh session store per restart	Both caught by failing tests

Database, store and session tests were written first and seen failing. The app-level tests were written after the wiring; the mutation checks confirm they can fail.

8. Files changed

File	Change
`server/src/db.ts`	New: open database, schema migration v1, `transaction()`
`server/src/store.ts`	New: accounts, disputes, evidence, access, demo mirror, audit
`server/src/auth.ts`	Sessions read and written in the database
`server/src/app.ts`	Routes read from the database; audit entries; atomic sign-in and sign-out
`server/src/index.ts`, `config.ts`	Open `server/data/disputedesk.sqlite` (or `DATABASE_PATH`) at startup
Test files	`db.test.ts` and `store.test.ts` new; `auth.test.ts` and `app.test.ts` extended
`.gitignore`, `README.md`, `CLAUDE.md`	Ignore the database folder; document the database and its rules
Unchanged	Client, `shared/`, root PayPal scripts, fixture files, headers, CSP, cookie, cross-site checks

9. Remaining risks

Risk	Impact	Next step
`node:sqlite` is still a release candidate in Node 24	Its API could change in a later Node	Pin Node 24; recheck on upgrade
Synchronous queries	Brief pauses under heavy load	Fine for one local process
One process per database	No horizontal scaling	A server database if ever hosted
Sign-in limit still in memory	A restart lifts a lockout	Move it to the database
Audit log has no retention	Grows until deleted	Set a retention period
File permissions not tightened on Windows	Other local users could read hashes	Restrict the folder if shared

10. Method and constraints kept

- **Skills used:** database-migrations, security-and-hardening, test-driven-development, api-and-interface-design, Ponytail. No plugins, MCP servers or agents were used.
- **Not done:** no live PayPal or Gemini calls, no new dependencies, no `.env` changes, nothing pushed.
- **Git:** commit `5145cbe` on `master`, 13 files. `reports/` stays untracked.